

Deep Reinforcement Learning for Bomberman

Eric Flämig, Mark Salzmann, Jesper Eggers

September 29, 2026

Code and dependencies. https://github.com/ericgoldclub/bomberman_rl. Neural agents require PyTorch in addition to the framework dependencies; Matplotlib produces the report figures. The new evaluations use Python 3.12, PyTorch 2.8.0, NumPy 2.2.6, Pygame 2.6.1, Matplotlib 3.10.6, and tqdm 4.67.1. Exact reproduction commands and hashes accompany the evaluation scripts.

Abstract

We develop two Double DQN Bomberman agents with different allocations of learned decisions and explicit guidance. A 3,000-state engine-based audit and 1,600 frozen-policy games distinguish escape feasibility from competitive performance. With final-agent weights fixed, replacing the RUEHL mask with the temporal mask adds 1.375 points per game against rule-based opponents; learned ranking adds 0.855 points beyond temporal masked random play while increasing self-destruction. A replayed opponent collision exposes why conditional escape search cannot guarantee safety. Together, these experiments connect engine-conditional escape feasibility with learned choices and competitive outcomes.

1 Introduction

Main author: Jesper Eggers

Bomberman makes a useful test of reinforcement learning because an action can create both an opportunity and a delayed threat to its own actor. Bombs clear terrain and eliminate opponents, but each placement creates a delayed hazard that the agent must anticipate. Navigation also changes as crates disappear, while opponents compete for coins and alter the available routes. A high shaped reward or successful play without opponents therefore need not predict competitive strength.

The six discrete actions suit value-based learning. Tabular Q-learning provides a conceptual starting point, but enumerating configurations of terrain, bombs, and opponents quickly becomes impractical. Neural action values offer generalization across configurations; direct policy optimization would be another possible approach. We specialize the value-based approach through engineered observations, dueling networks, reward shaping, and masks.

Our project implements two agents based on Double DQN and dueling networks. RUEHL uses a compact learned representation and comparatively simple action constraints. The final agent combines a larger representation with explicit temporal survival searches; its name does not denote the submitted player, which was RUEHL. The central experimental question is how learned action preferences and safety guidance contribute to their deployed behavior. Alongside the existing development results, we test the masks against engine-derived continuations and evaluate frozen final-agent weights in a controlled action-selection experiment. This separates constraint quality, learned ranking, and competitive score without adding another architecture or training campaign.

We distinguish engine-conditional escape feasibility, learned choices within the constraints, and competitive outcomes. Temporal guidance and learned ranking both add score in the tested deployment configurations, while learned selection increases self-destruction relative to masked random play. A replayed opponent collision explains why conditional reachability remains weaker than competitive safety.

2 Background

Main author: Mark Salzmann

2.1 Bomberman as a reinforcement learning problem

Main author: Mark Salzmann

At each time step, the agent observes the game, chooses an action, and receives feedback through game events. Its action set is

$$\mathcal{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}, \text{WAIT}, \text{BOMB}\}.$$

An episode corresponds to a round of play from the agent’s perspective and ends when it dies or the round terminates. The configured board has 17×17 tiles, with a maximum of four agents and 400 steps per round. The aim of the game is to collect coins and eliminate opponents. Coins yield points immediately, whereas placing a bomb can produce a benefit only after its timer expires. The agent must therefore consider future consequences rather than only the next reward.

A Markov decision process provides the value-learning framework, with states, actions, transitions, rewards, and discount factor γ . The complete environment state includes hidden coin locations and opponent memory, while the callback exposes only an observation. Opponents’ decisions also affect transitions. We use engineered observations and limited action history as an approximate state representation, rather than claiming that the network input is Markov sufficient. To make optimal decisions, the next action should be chosen based on the expected future rewards, considering both immediate and delayed consequences, captured by the action-value function defined as

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]. \quad (1)$$

Here, r_{t+1} is the learning reward, T is the episode’s terminal time, and π is the policy. A larger discount factor gives delayed outcomes greater weight. For example, a useful bomb placement may deserve a high value even if the agent must first move away and wait for the explosion.

2.2 From Q-learning to deep Q-learning

Main author: Mark Salzmann

A deep Q-network (DQN) uses a neural network to approximate the action-value function, allowing it to generalize across similar states and handle large state spaces. For a given input state s the network outputs a vector of action values $Q(s, a, \theta)$ for all possible actions $a \in \mathcal{A}$.

An illustrative squared-TD semi-gradient update changes parameters θ as follows. The implemented agents instead use Smooth L1 loss and AdamW, as described in their optimization sections

$$\theta_{t+1} = \theta_t + \alpha \left(Y_t^Q - Q(S_t, A_t, \theta_t) \right) \nabla_{\theta} Q(S_t, A_t, \theta_t), \quad (2)$$

where α is the learning rate. The target is defined below, with $d_{t+1} = 1$ on agent death or round termination and zero otherwise; terminal transitions do not bootstrap

$$Y_t^Q = r_{t+1} + \gamma(1 - d_{t+1}) \max_a Q(S_{t+1}, a, \theta_t). \quad (3)$$

Previous steps get stored in the *experience replay* buffer, which allows the network to sample minibatches of past transitions for training, reducing the correlation between consecutive updates. Furthermore the target network with parameters θ^- is used to compute the target values, and it is updated every τ steps from the online network's parameters such that $\theta^- = \theta_t$. The target used in the update then becomes

$$Y_t^Q = r_{t+1} + \gamma(1 - d_{t+1}) \max_a Q(S_{t+1}, a, \theta^-), \quad (4)$$

where θ^- are the parameters of the target network.

2.3 Double DQN: separating selection and evaluation

Main author: Mark Salzmann

The maximum in Equation (4) selects and evaluates an action using the same estimates. The issue with this method is that is prone to overestimation of action values, leading to bias in the learned Q-values. Van Hasselt et al. show that this overestimation also occurs with deep networks and propose Double DQN to reduce it [5].

They therefore propose the Double DQN approach, which uses the online network with parameters θ to select the next action and the target network with parameters θ' to evaluate that action. In both implementations, gradient updates change the online network and a hard copy updates the target network. RUEHL measures its copy interval in gradient updates, whereas the final agent measures it in regular training-transition steps; the two networks do not randomly exchange roles. The double DQN target is defined as

$$Y_t^{\text{DoubleDQN}} = r_{t+1} + \gamma(1 - d_{t+1}) Q\left(s_{t+1}, \arg \max_{a \in \mathcal{A}_m(s_{t+1})} Q(s_{t+1}, a, \theta_t); \theta'\right), \quad (5)$$

Here, $\mathcal{A}_m(s)$ is the nonempty set admitted by the action mask. Action selection in the $\arg \max$ uses the online parameters θ_t , while the target parameters θ' evaluate the selected action. This separation is intended to reduce overestimation; it does not guarantee an unbiased estimate.

In Bomberman, a relevant motivation is to avoid overvaluing risky actions whose apparent payoff is based on inaccurate predictions. This motivates Double DQN in our setting, although improved safety or competitive performance would require a controlled comparison against an otherwise identical learning algorithm.

3 Project planning

Main author: Jesper Eggers

Each group member primarily owns their report sections. The project maintains two agent implementations and a staged training workflow: transfer from saved weights, alternate training and greedy evaluation, then archive the best model and parameter snapshot. Training logs record two 30,000-round RUEHL continuation stages, lasting 8:47:59 and 8:03:33. Their 180/20 schedule nominally allocates 27,000 rounds to training and 3,000 to evaluation per stage; these are played-round budgets, not counts of optimization updates.

The final evaluation stage freezes the final network and varies its action-selection components. This separates model development from assessing what each deployed component contributes, while keeping the evaluation workload finite before the report deadline. The paired-game design gives each configuration the same initial conditions, and the analysis connects game scores to survival and action-selection behavior. The organizational lesson is to preserve source revisions, model hashes, per-episode evaluations, and hardware records together at each milestone.

4 Methods

Main authors: Mark Salzmann, Eric Flämig, Jesper Eggers

4.1 RUEHL agent

Main author: Mark Salzmann

4.1.1 Input representation and feature engineering

Main author: Mark Salzmann

The function `state_to_features` converts the observation into seven spatial maps and ten scalar values. The board input has shape $7 \times 17 \times 17$ and uses game information directly rather than rendered screen pixels. Table 1 summarizes its channels.

Channel	Encoding
Walls	Binary map of indestructible walls.
Crates	Binary map of destructible crates.
Coins	Binary map of currently visible coins.
Own position	Binary map of the agent’s tile.
Opponents	Binary map marking each opponent’s tile.
Bomb timers	At each bomb tile, $(t_b + 1)/5$ for its remaining timer t_b ; zero elsewhere.
Combined danger	Maximum of the active-explosion indicator and $(5 - t_b)/5$ across bombs whose blast includes a tile.

Table 1: Spatial features implemented in `callbacks.py`.

The combined danger map already merges active explosions with prospective bomb threats. The denominators above use the configured bomb timer of four; this channel encodes urgency rather than raw explosion duration.

The ten scalar inputs comprise four state quantities and a six-entry previous-action encoding. They contain the agent’s bomb availability, currently visible coin count, remaining opponent

count, and current step, with the normalization shown in Table 2. The round number is not an input.

Channel	Encoding
Bomb availability	Binary value indicating if the agent can place a bomb.
Visible coins	Number of currently visible coins divided by the configured total coin count.
Remaining opponents	Number of opponents divided by $\text{MAX_AGENTS} - 1 = 3$.
Current step	Current step divided by $\text{MAX_STEPS} = 400$.
Previous action	Six-entry one-hot encoding of the previous action; initialized to <code>WAIT</code> .

Table 2: Scalar features implemented in `callbacks.py`.

These scalars summarize the current observation and previous action. Own and opponent positions remain spatial features; hidden coins are not counted until they become visible.

4.1.2 Neural network architecture

Main author: Mark Salzmann

Given the combination of spatial and scalar features, it is reasonable to use a network architecture that processes these two types of inputs separately before combining them for decision making. The spatial branch contains four convolutional layers with ReLU activations. Dilated convolutions expand the area visible to a filter without increasing its number of weights, while the third convolution reduces the spatial resolution. Table 3 gives the dimensions derived from `Networks.py` with the batch dimension is omitted.

Component	Operation	Output shape
Board input	Seven spatial channels	$7 \times 17 \times 17$
Convolution 1	3×3 , 32 filters, stride 1, padding 1	$32 \times 17 \times 17$
Convolution 2	3×3 , 32 filters, dilation 2, padding 2	$32 \times 17 \times 17$
Convolution 3	3×3 , 32 filters, stride 2, padding 1	$32 \times 9 \times 9$
Convolution 4	3×3 , 32 filters, dilation 2, padding 2	$32 \times 9 \times 9$
Flatten	Flatten the board features	2592
Scalar branch	Linear $10 \rightarrow 32$, ReLU	32
Concatenation	Join spatial and scalar features	2624
Shared layers	Linear $2624 \rightarrow 128 \rightarrow 128$, ReLU after each	128
Value head	Linear $128 \rightarrow 1$	1
Advantage head	Linear $128 \rightarrow 6$	6

Table 3: Architecture of the RUEHL network for the configured board size.

Each output of the last convolution has a theoretical receptive field of 17×17 input tiles, including padding near the boundary. The flattened features subsequently connect to shared fully connected layers.

Complementing the double DQN strategy, one can employ a dueling network architecture to separately estimate state values and action advantages, which can lead to more stable learning and improved policy evaluation. In the dueling DQN architecture, the action-value function is not represented directly by a single network output. Instead, the network decomposes the estimate of $Q(s, a)$ into two components: a state-value function $V_\theta(s)$ and an action-dependent advantage function $A_\theta(s, a)$ [6]. The state-value function represents the overall quality of being in state s , independently of which action is selected, whereas the advantage function describes the relative benefit of choosing a particular action a in that state.

A naive decomposition of the form

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a)$$

is not identifiable. In particular, an arbitrary constant c can be added to the value function and subtracted from all advantage values without changing Q_θ :

$$V_\theta(s) + c + (A_\theta(s, a) - c) = V_\theta(s) + A_\theta(s, a).$$

Consequently, the network would have no unique way of determining which part of the predicted action value should be represented by V_θ and which part should be represented by A_θ .

To remove this ambiguity, the dueling architecture combines the two streams by subtracting the mean advantage over all six action outputs, before masking,

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{b \in \mathcal{A}} A_\theta(s, b). \quad (6)$$

This centers the effective advantage values around zero. An important consequence of this parameterization is obtained by averaging $Q_\theta(s, a)$ over all actions:

$$\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q_\theta(s, a) = V_\theta(s).$$

Thus, under this aggregation rule, the value stream represents the mean predicted action value in a state, while the centered advantage stream determines how the individual action values differ from this common baseline. In particular, because both $V_\theta(s)$ and $\bar{A}_\theta(s)$ are independent of the selected action,

$$\arg \max_a Q_\theta(s, a) = \arg \max_a A_\theta(s, a).$$

Hence, the advantage stream determines the relative ranking of actions, whereas the value stream controls the overall level of their Q-values.

The motivation for this decomposition is that in many states the precise choice of action has only a small effect on the expected return. A conventional DQN must nevertheless estimate a separate $Q(s, a)$ for every action. The dueling architecture allows the network to learn features that characterize whether a state is generally favorable or unfavorable through $V_\theta(s)$, while using the advantage stream only to represent differences between actions. This can make value estimation more efficient in states where several actions have similar consequences [6].

The two quantities are not trained using separate value and advantage targets. Instead, they are combined according to Eq. (6) to obtain $Q_\theta(s, a)$, and the usual temporal-difference loss is applied to this resulting Q-value. Gradients then propagate through the aggregation operation into both the value and advantage streams. The decomposition is therefore an architectural parameterization of the Q-function rather than the introduction of two independently supervised learning objectives.

4.1.3 Action masking and hand-designed guidance

Main author: Mark Salzmann

The mask is an important part of an effective policy. It first rejects movement into walls, crates, bombs, or opponents. It then filters movement and waiting on active explosions or blast tiles of bombs whose timers are at most one. Bomb placement is allowed only when a bomb is available, its blast could hit a crate or opponent, and a static breadth-first search finds a tile outside its blast within the configured bomb timer.

If no action survives the danger filter, the code falls back to the earlier legal-action mask. If that is also empty, the model chooses `WAIT`. This is therefore a heuristic filter, not a guarantee of survival. Its static escape search does not predict moving opponents or jointly simulate other bombs over time.

This guidance reduces the exploration burden, but it also contributes to the observed behavior independently of learned Q-values. Results should therefore describe the complete network-plus-mask policy. The Double DQN update in `train.py` also applies the next-state action mask before the online network selects its maximizing action. The target network then evaluates that selected action.

4.1.4 Reward shaping

Main author: Mark Salzmann

The main goal of the agent is to maximize game score at the end of the game, by collecting coins, eliminating opponents, given by

$$S = N_{\text{coins}} + 5N_{\text{opponents killed}}. \quad (7)$$

However, starting from these sparse and delayed objectives alone makes it difficult for the agent to learn effective strategies. Reward shaping addresses this problem, by giving intermediate rewards and punishments that guide the agent’s behavior before the final game outcome is known. For the `ruehl_based_agent`, equivalent movements receive the same reward irrespective of absolute direction. This encourages consistency with board symmetries, but neither these reward magnitudes nor standard convolutions guarantee rotation or reflection equivariance. Additional RUEHL shaping is derived from observed transitions and events, including waiting, danger, and movement reversal. These terms guide learning without establishing policy invariance. Furthermore, one can differentiate between main rewards, connecting to significant game events, and shaping rewards, which serve as fine tuning of the agent’s behavior. The shaping rewards are an order of magnitude smaller than the main rewards.

Main rewards The main contribution, stored under `major`, assigns values to selected game events. Collecting a coin provides a baseline reward, typically 1 point, in accordance with the actual game score. Other rewards will be scaled relative to this baseline. A separate Self-destruction penalty is useful in competitive play. This expresses the trade-off between gaining points and retaining the opportunity to act. Penalizing only death would encourage excessive caution, meanwhile rewarding only attacks would provide insufficient feedback about their danger. Their combination is intended to encourage productive risks with an escape plan.

Shaping penalties The separate shaping terms discourage waiting, entering an active explosion, and, in the competitive profile, immediately reversing movement direction. Their purpose is to

discourage inactivity and repeated back-and-forth motion without making these actions impossible. Waiting for an explosion to clear or reversing direction to escape can be necessary, so these penalties should remain small relative to important outcomes.

4.1.5 Task-specific reward profiles

Main author: Mark Salzmann

We identified three substantially different challenges, for which we designed corresponding reward profiles: `coin`, `loot_exploration`, and `killer`. Determining the appropriate distribution of rewards was a challenging task that required long trial and error processes. The current values are summarized in Table 4.

Event or parameter	<code>coin</code>	<code>loot_exploration</code>	<code>killer</code>
Coin collected	1.0	1.0	1.0
Coin revealed	0.1	0.5	0.5
Crate destroyed	0	0.35	0.15
Bomb placed to hit a crate	0	0.05	0.02
Bomb placed to threaten an opponent	0	0	0.75
Opponent killed	6.5	6.5	6.0
Killed by another agent	-1.0	-1.0	-2.0
Self-destruction	-2.0	-2.0	-6.0
Bomb exploded	0	0	0.15
Round survived	0	0	0.75
Waiting (shaping)	-0.08	-0.01	-0.05
Danger (shaping)	-0.02	-0.04	-0.10
Reversing direction (shaping)	0	0	-0.03
Shaping limit c	0.05	0.05	0.15
Step cost λ	0.008	0	0.002

Table 4: Reward profiles in `train.py`. Event values are summed per transition; only the combined shaping contribution is clipped. A zero denotes an event with no assigned reward in that profile.

Coin collection. The `coin` profile rewards collection and discourages waiting through both a shaping penalty and a small step cost. It gives no direct incentive for crate destruction or bomb placement, focusing the initial task on navigation. Opponent-kill rewards are inherited from the shared reward table but have no effect in a scenario without opponents.

Crate exploration. The `loot_exploration` profile rewards destroying crates, revealing coins, and placing bombs whose blast reaches a crate. These intermediate rewards provide feedback before the agent eventually collects a coin. Removing the step cost and reducing the waiting penalty allows more time for bomb placement and escape sequences, while the self-destruction penalty retains an incentive to survive.

Competitive play. The `killer` profile emphasizes successful attacks and survival. A bomb that currently threatens an opponent earns an immediate reward of 0.75, while a confirmed kill earns 6.0. Self-destruction costs 6.0, substantially more than death caused by another agent. Smaller crate rewards preserve an incentive to open routes, and surviving the round earns 0.75. The bomb-placement event checks the current blast geometry, not whether the opponent will actually be hit. If both opponents and crates are in range, only the opponent-targeting placement event is added.

4.2 Final agent

Main author: Eric Flämig

It should be noted that the final agent is only final by name and was not the submitted agent for the competition.

The final agent is based on the same Double DQN learning framework and broadly similar staged objectives as the Ruehl based agent, but with some modifications to the input representation and reward shaping. The intention behind the final agent was to explore the impact of a bigger Network architecture, and to investigate the effect of a different reward shaping strategie and the use of a more guided action mask on the agent’s performance.

Compared to the Ruehl based agent, the final agent uses a larger network with more convolutional layers and filters, allowing it to capture more complex spatial relationships in the game environment. Additionally, the final agent incorporates a more sophisticated reward shaping strategy that takes into account not only the immediate rewards but also the long-term consequences of actions, encouraging the agent to develop more strategic behaviors. It also incorporates a more guided action mask that restricts the agent’s actions based on the current game state, helping it to avoid dangerous situations and make more informed decisions.

4.2.1 The Features

Main author: Eric Flämig

The features used for this Network are one of the main differences between the Ruehl based agent and the final agent.

The spatial input consists of fourteen 17×17 feature maps containing information such as walls, crates, coins, agent positions, and bomb-related danger. Each one of these maps is designed to capture only one spatial aspect of the game state, which resulted in 14 different channels of information which are summarized in Table 5. The twenty-two scalar features in Table 6 supply directional, temporal, and opponent-related guidance to the network. They differ substantially from RUEHL’s compact scalar input. Their sample-efficiency and generalization effects are design hypotheses, not isolated empirical findings.

4.2.2 Neural Network Architecture

Main author: Eric Flämig

The final agent’s network architecture is similar to the Ruehl based agent, but with some modifications to the convolutional layers and the fully connected layers.

It is based on the same two headed logic, where a CNN evaluates the spatial features and a fully connected network evaluates the scalar features. The outputs of both networks are then concatenated and passed through a series of fully connected layers to produce the final Q-values for each action. In theory a simple CNN should be sufficient for this task, but we decided to add a fully connected network to potentially speed up the training process.

During development, earlier deeper versions were reported to train slowly and make weak use of scalar information. These observations motivated preserving a scalar embedding and adding residual blocks and smooth activations. They were not measured in a common-budget ablation and do not establish the contribution of each modification. The current HybridDQN is summarized below. The final agent’s HybridDQN combines a convolutional neural network with

Channel	Feature	Encoding
0	Walls	Binary map of indestructible walls.
1	Crates	Binary map of destructible crates.
2	Coins	Binary map of visible coins.
3	Own position	One-hot encoding of the agent’s current tile.
4	Useful bomb positions	Binary map of free tiles where placing a bomb could hit a crate.
5	Bomb timers	Remaining timer divided by 4 at each bomb’s position; zero elsewhere.
6	Bomb danger	Binary map of tiles within the blast area of any existing bomb, regardless of its timer.
7	Active explosions	Binary map indicating where <code>explosion_map</code> > 0.
8	Free terrain	Binary map indicating where <code>field</code> = 0; tiles may still contain bombs or agents.
9	Opponent positions	Binary map of opponents’ current tiles.
10	Explosion urgency	$1 - t_{\min}/4$ at threatened tiles, where $t_{\min}$ is the smallest timer among bombs whose blasts cover that tile; zero elsewhere.
11	Safe neighboring tiles	Binary map of adjacent tiles for which the survival search finds a surviving continuation after moving there.
12	Own hypothetical blast	Binary map of tiles affected by a hypothetical bomb at the agent’s current position.
13	Opponents’ hypothetical blasts	Binary map of tiles affected by hypothetical bombs at opponents’ current positions.

Table 5: The final agent’s fourteen spatial input channels. Each channel has dimensions 17×17 . Channel indices follow the implementation in `callbacks.py`.

a fully connected network to process two complementary representations of the game state. The convolutional branch extracts spatial patterns from the board, while the fully connected branch processes engineered features that describe relevant game situations explicitly. Their outputs are combined and passed to a dueling architecture, which estimates a shared state value and the relative advantages of individual actions.

The first convolution transforms the 14 spatial feature maps into 32 feature maps while preserving their spatial dimensions. Two subsequent convolutional stages increase the number of channels to 64 and 96, respectively. These stages use a stride of two, reducing the spatial resolution first to 9×9 and then to 5×5 . This lets the network combine information across increasingly large regions of the board while limiting the number of features passed to the fully connected layers. Each of these convolutions is followed by group normalization with eight groups and a SiLU activation. The SiLU function was chosen, because with ReLU we previously observed a significant number of dead neurons in the fully connected layers, which we hoped to prevent with the SiLU activation.

Each convolutional stage also contains a residual block. A residual block processes its input through two additional convolutions and then adds the original input to their output. For an input tensor x , the block computes

$$ResBlock(x) = GN_2(Conv_2(SiLU(GN_1(Conv_1(x))))), \quad (8)$$

followed by

$$y = SiLU(x + ResBlock(x)). \quad (9)$$

Both convolutions use 3×3 kernels, stride one, and padding one. They preserve the number of channels and the spatial dimensions, so the input x and the transformed features $F(x)$ can be added element by element. Consequently, the residual blocks preserve the shapes $32 \times 17 \times 17$,

Index	Feature	Encoding
0	Coin direction, x	Suggested next step's x -offset toward a coin, divided by 17.
1	Coin direction, y	Suggested next step's y -offset toward a coin, divided by 17.
2	Crate-target direction, x	Suggested next step's x -offset toward a useful bomb position, divided by 17.
3	Crate-target direction, y	Suggested next step's y -offset toward a useful bomb position, divided by 17.
4	Crate within bomb range	Binary indicator that a bomb at the current position could hit a crate.
5	Bomb availability	Integer value of <code>bombs_left</code> .
6	Current danger	Binary indicator that the current tile lies in an existing bomb's blast area or an active explosion.
7	Safe movement available	Binary indicator that at least one movement action has a surviving continuation.
8	Previous position, x	$(x_{\text{previous}} - x)/17$.
9	Previous position, y	$(y_{\text{previous}} - y)/17$.
10	Two positions back, x	$(x_{\text{two back}} - x)/17$.
11	Two positions back, y	$(y_{\text{two back}} - y)/17$.
12	Round progress	Current step divided by 400.
13	Closest opponent distance	Minimum Manhattan distance to an opponent, divided by 17; set to 2 if no opponents remain.
14	Opponent within bomb range	Binary indicator that a bomb at the current position could hit an opponent.
15	Survival after bomb placement	Binary indicator that the search finds a surviving path after placing a bomb and remaining on the current tile for that action.
16	Potential attacking opportunity	Binary conjunction of opponent within bomb range and survival after bomb placement.
17	Predicted trapped opponent	Binary indicator that an opponent lies in the proposed bomb's blast area and the search finds no escape for that opponent.
18	Opponent direction, x	Suggested next step's x -offset toward an opponent, divided by 17.
19	Opponent direction, y	Suggested next step's y -offset toward an opponent, divided by 17.
20	Remaining opponents	Number of opponents divided by 3.
21	Current score	Raw game score without normalization.

Table 6: The final agent's twenty-two scalar input features. Here, (x, y) is the agent's current position. Direction features describe the next step suggested by the path search. Historical positions refer to recorded position changes, with consecutive repetitions omitted; missing history entries are encoded as zero. Indices follow the implementation in `callbacks.py`.

Component	Operation	Output shape
Board input	Fourteen spatial channels	$14 \times 17 \times 17$
Convolution 1	3×3 , 32 filters, stride 1; GN, SiLU	$32 \times 17 \times 17$
Residual block 1	Two 3×3 convolutions, 32 filters; skip connection	$32 \times 17 \times 17$
Convolution 2	3×3 , 64 filters, stride 2; GN, SiLU	$64 \times 9 \times 9$
Residual block 2	Two 3×3 convolutions, 64 filters; skip connection	$64 \times 9 \times 9$
Convolution 3	3×3 , 96 filters, stride 2; GN, SiLU	$96 \times 5 \times 5$
Residual block 3	Two 3×3 convolutions, 96 filters; skip connection	$96 \times 5 \times 5$
Flatten	Flatten the spatial features	2400
Scalar input	Twenty-two engineered features	22
Scalar branch	Linear $22 \rightarrow 64 \rightarrow 64$, LN and SiLU after each layer	64
Concatenation	Join spatial and scalar features	2464
Shared layers	Linear $2464 \rightarrow 512 \rightarrow 256$, LN and SiLU after each layer	256
Value head	Linear $256 \rightarrow 128 \rightarrow 1$, SiLU after the hidden layer	1
Advantage head	Linear $256 \rightarrow 128 \rightarrow 6$, SiLU after the hidden layer	6
Dueling output	$Q(s, a) = V(s) + A(s, a) - \frac{1}{6} \sum_{b=1}^6 A(s, b)$	6

Table 7: Architecture of the final agent’s **HybridDQN**. The batch dimension is omitted. GN denotes group normalization with eight groups; LN denotes layer normalization. All convolutions use padding 1. Each residual block uses stride 1 and applies convolution–GN–SiLU–convolution–GN, followed by addition of the block input and a final SiLU activation.

$64 \times 9 \times 9$, and $96 \times 5 \times 5$, respectively. The resolution changes occur in the strided convolutions between the blocks.

The addition of x to the output of the residual block allows the convolutional path to learn corrections or additions to an existing representation. For example, a block can refine features describing nearby crates by incorporating surrounding escape routes. During backpropagation, the addition also provides a direct path for gradients alongside the path through the convolutional layers, making it easier to train a deeper spatial encoder. This was implemented to allow the network to learn from a pretrained starting point, and to improve the learning of more complex spatial relationships.

The fully connected branch takes the 22 engineered features as input and processes them through two linear layers with 64 hidden units each. Each layer is followed by layer normalization and a SiLU activation. The output of this branch is a 64-dimensional feature vector that captures relevant scalar information about the game state.

The final agent uses the dueling decomposition described for the Ruehl based agent, combining its value and advantage outputs according to Equation (6).

In the final agent, both heads receive the shared 256-dimensional representation. Each head contains a separate fully connected hidden layer with 128 units and a SiLU activation. A final linear layer produces one value estimate in the value head and six action-specific outputs in the advantage head.

Compared with the direct output layers of the Ruehl network, these additional hidden layers allow each head to apply its own nonlinear transformation to the shared representation before producing its output. The aggregation rule remains unchanged.

The architecture increases representational capacity and preserves a separate scalar embedding. Whether its residual blocks, normalization, or activation choices improve learning requires controlled comparisons.

4.2.3 The Action Mask

Main author: Eric Flämig

One of the main differences between the Ruehl based agent and the final agent is the use of a more guided action selection through the action mask. The intent was to reduce the amount of possible scenarios to only the ones that were legal moves and that did not lead to an immediate suicide of the agent. The reasoning behind that restrictive approach, was to reduce the training time.

The final agent uses a hand-designed action mask to restrict action selection according to physical feasibility, predicted survival, and the usefulness of bomb placement. For each state, the mask assigns a Boolean value to each of the six actions.

The survival check performs a breadth-first search, which was implemented from the rule based agent, over positions and future time steps, using a horizon of six steps. The first action is fixed to the candidate being evaluated; subsequent steps may contain any of the four movement actions or waiting. A candidate passes the check if at least one continuation survives the entire search horizon. The search accounts for walls, crates, bomb occupancy, bomb timers, and active explosions. Consequently, it can reject a move that is safe immediately but leads to an unavoidable explosion a few steps later.

For bomb placement, the search inserts a hypothetical bomb at the agent’s current position. The agent remains on that tile during the placement action, and the search then checks whether a surviving continuation exists. Bomb placement additionally requires an available bomb and a blast area that could reach at least one crate or opponent. Table 8 summarizes the rules.

Action or stage	Rule
Movement	Allow the action if the destination is traversable and the survival search finds a continuation lasting the full six-step horizon.
WAIT	Allow waiting if remaining on the current tile for the first step permits a surviving continuation.
BOMB	Allow placement if a bomb is available, its blast could reach a crate or opponent, and the survival search succeeds with the additional bomb included.
First fallback	If the preceding rules reject every action, allow movement to adjacent tiles inside the board that contain no wall, crate, bomb, or opponent. Explosion danger is disregarded at this stage; waiting and bomb placement remain excluded.
Final fallback	If no movement action passes the first fallback, allow WAIT, even if survival cannot be established.

Table 8: Action-mask rules of the final agent. The fallback stages are applied only when the preceding stage leaves no permitted action.

The mask is applied during both exploration and exploitation. Exploration samples uniformly from the permitted actions. For greedy selection, excluded actions receive a temporary value of $-\infty$:

$$\tilde{Q}(s, a) = \begin{cases} Q(s, a), & \text{if the mask permits } a, \\ -\infty, & \text{otherwise.} \end{cases} \quad (10)$$

The agent then selects the action maximizing $\tilde{Q}(s, a)$. During Double DQN training, the next-state mask also restricts the online network’s action selection when constructing the bootstrap target; the target network evaluates the selected action.

The survival prediction uses a simplified model: terrain remains fixed during the search, opponents’ current positions block only the first step, and future opponent movements and new bomb

placements are not predicted. Together with the fallback rules, these assumptions mean that the mask cannot guarantee survival. It provides explicit guidance that contributes to the agent’s behavior alongside the learned Q-values, while restricting exploration to actions admitted by its rules.

4.2.4 The Reward Shaping

Main author: Eric Flämig

We aimed to align final-agent shaping with represented observations. The implementation also uses transition events and limited tracked history, including the most recent own bomb; reward observability is a design preference rather than a requirement that every reward condition occupy its own input feature.

The final agent combines rewards for game outcomes with smaller rewards for intermediate behavior. These intermediate signals provide feedback before an action sequence produces a measurable outcome. For example, destroying a crate and collecting the coin underneath requires the agent to approach a suitable position, place a bomb, escape its blast, and return after the explosion. Reward shaping provides feedback at several points in this sequence.

The learning reward is constructed by summing the contributions of the events associated with a transition. Multiple contributions can therefore occur simultaneously: collecting a coin can coincide with movement-related rewards and the step penalty. Events without an assigned reward contribute zero. The current reward configuration is summarized in Table 9.

Collecting coins and eliminating opponents receive substantially larger rewards than individual shaping events. This gives completed objectives greater weight than intermediate progress. Crate destruction and coin revelation receive smaller rewards because they create opportunities for later collection. These values define a learning objective with its own relative priorities: an opponent elimination is worth approximately $25/7 \approx 3.57$ collected coins.

Death receives a substantial negative contribution. The implementation distinguishes self-destruction, penalized by -20 , from other deaths, penalized by -15 . Since self-destruction also generates the general death event, the reward function explicitly applies only the self-destruction penalty in this case. This prevents an unintended combined penalty of -35 . Other rewards generated during the same transition still contribute to the total.

Bomb-placement shaping distinguishes between predicted usefulness and confirmed success. A placement receives $+0.75$ if the agent has a predicted escape route and an opponent within the blast area is predicted to have no escape. If an opponent is threatened but no trap is predicted, the reward is $+0.30$. A placement targeting a crate receives $+0.20$ when an escape is predicted and no opponent is threatened. These conditions are evaluated in priority order, so a single placement receives only one placement classification. Placements failing these checks receive -0.50 . The larger reward for an actual elimination is awarded separately when the outcome occurs.

The agent also tracks its most recent bomb placement. Leaving that bomb’s blast area produces a reward of $+0.30$. Remaining inside the area produces a penalty of -1.00 only when the survival search finds no surviving continuation. This condition allows temporary movement within the blast area while an escape remains possible.

Movement shaping uses changes in Euclidean distance to coins, opponents, and positions from which a bomb could destroy a crate. Decreasing the distance produces a small positive reward, while increasing it produces a negative contribution. The crate-related target is therefore a useful placement position rather than the crate’s occupied tile. These signals provide frequent

Event or condition	Reward
<i>Game outcomes</i>	
Opponent eliminated	+25.00
Coin collected	+7.00
Round survived	+1.00
Crate destroyed	+0.10
Coin revealed	+0.05
Suicide	-20.00
Got killed by an opponent	-15.00
<i>Bomb placement and escape</i>	
Bomb predicted to trap an opponent	+0.75
Bomb threatening an opponent without a predicted trap	+0.30
Bomb targeting a crate without threatening an opponent	+0.20
Bomb placement failing the preceding checks	-0.50
Leaving the tracked own bomb's blast area	+0.30
Remaining in that blast area without a predicted survival path	-1.00
<i>Movement guidance</i>	
Moving closer to a coin	+0.05
Moving farther from a coin	-0.05
Moving closer to an opponent	+0.05
Moving farther from an opponent	-0.05
Moving closer to a useful crate-bombing position	+0.03
Moving farther from a useful crate-bombing position	-0.03
<i>Behavioral penalties</i>	
Invalid action	-1.00
Waiting	-0.05
Returning to the position recorded two steps earlier	-0.50
Step cost on regular training transitions	-0.01

Table 9: Reward configuration of the final agent. Positive bomb-placement rewards require a predicted survival path. Bomb-placement categories are mutually exclusive, and the self-destruction penalty replaces the ordinary death penalty.

directional feedback, although Euclidean distance does not account for obstacles or the length of a traversable route. Equal forward and backward magnitudes remove a simple undiscounted oscillatory incentive. They do not establish policy invariance. The standard potential-based guarantee requires $F(s, s') = \gamma\Phi(s') - \Phi(s)$ under its assumptions [3]. Our distance, waiting, and placement bonuses are heuristic objectives and require evaluation under official score.

Waiting and invalid actions receive penalties, and regular training transitions include a small step cost. The position-history check also penalizes a return to the position recorded two steps earlier. This discourages back-and-forth movement, but the same condition can also be satisfied by repeated waiting.

As an illustrative example, a sequence involving a predicted-safe crate-targeting placement, leaving the blast area, destroying one crate, revealing one coin, and collecting that coin contributes

$$0.20 + 0.30 + 0.10 + 0.05 + 7.00 = 7.65.$$

This is the undiscounted sum of those event contributions; movement rewards, step costs, and other events would modify the sequence’s total reward.

The shaping terms also introduce trade-offs. A necessary detour may increase the distance to a target, and retreating from an opponent may be essential for survival. Waiting can likewise be appropriate while an explosion clears. Furthermore, repeated shaping rewards can accumulate because their combined contribution is not clipped. Evaluation should therefore examine game score, coin collection, eliminations, and survival alongside learning reward. Controlled comparisons with individual shaping terms removed would be needed to establish which components improve performance.

4.3 Experimental evaluation methodology

Main author: Jesper Eggers

The agents differ in representation, network, mask, rewards, and training history. Whole-policy comparisons therefore cannot isolate the cause of a score difference. We distinguish complete-agent performance from controlled component interventions and engine-based constraint diagnostics.

4.3.1 Separate learning reward, selection score, and game performance

Main author: Jesper Eggers

For episode i , the primary outcome is the official score

$$S_i = C_i + 5K_i,$$

where C_i counts collected coins and K_i counts credited opponent kills. Crate destruction, survival, and threatening an opponent matter strategically, but do not directly increase this score. We record survival, bomb use, predicted threats, self-destruction, and episode length as diagnostics, so that a score change can be interpreted in terms of behavior. For competitive games, the focal agent’s score should also be compared with its opponents’ scores. We define score-leader share by dividing one unit of credit equally among agents tied for the highest final score. Games continue after the focal agent dies, so scores include later credits from its bombs and opponents’ subsequent play. All four agents, including those eliminated, enter the ranking. This measures relative score, not last-survivor success.

The RUEHL checkpoint-selection function in the competitive profile is

$$J = \overline{S} + 5\overline{K} - 5\widehat{p}_{\text{self}} = \overline{C} + 10\overline{K} - 5\widehat{p}_{\text{self}}.$$

Thus, checkpoint selection assigns a kill twice its official point value relative to a coin. The selection objective J and the final agent’s shaped reward both differ from tournament score and require distinct labels.

Similarly, RUEHL’s recorded “completion” event requires that the focal agent has collected the configured total number of coins and that no opponents remain. In a competitive game, an opponent can collect even one coin and make this criterion unattainable. A zero completion rate therefore does not establish a zero survival rate or a zero win rate. This distinction is necessary when interpreting the curriculum table.

4.3.2 Independent tests and controlled comparisons

Main author: Jesper Eggers

The twenty-game greedy evaluations used for checkpoint selection are validation measurements. Repeatedly choosing the largest score also selects favorable noise; a monotone best-so-far curve need not show convergence. Fresh games after freezing the checkpoint estimate subsequent performance.

We evaluated the existing final-agent checkpoint in the classic scenario, with zero exploration, no learning updates, and 200 fresh episodes per configuration. The checkpoint was selected by the existing callback’s default filename before observing these results, rather than by test-score selection. Strict tensor verification and warning-to-error handling exclude silent random fallback. No architecture, reward, or trained weight was changed.

The four configurations cross the final network’s greedy Q-value ranking or uniform random selection with the final temporal mask or RUEHL mask. Both learned configurations use the final agent’s fourteen spatial and twenty-two scalar inputs. Thus, “Final Q + RUEHL mask” is a deployment variant of the final network, not the trained RUEHL agent. Each configuration faces three rule-based opponents and, separately, a mixed group containing one rule-based, one coin-collector, and one peaceful agent. This yields 1,600 games.

The mask intervention changes the complete implementation, including escape checks, occupancy handling, and fallback rules; it does not isolate lookahead alone. The final network also retains its temporal-survival input features under both masks. Consequently, the mask swap removes neither all explicit safety reasoning nor all engineered guidance.

All four configurations start each paired episode from the same arena and spawn assignment, with board seeds separated from checkpoint selection. Merely launching two multi-round runs with the same seed is insufficient: the environment also uses its random generator to permute action order, so different episode lengths consume different numbers of random draws and alter later boards. We reset the board generator before each round, verify hashes of the arena, hidden-coin placement, and spawns, then use a separate seeded generator for action order. Each agent has isolated Python and NumPy random streams. Episodic position history is cleared between games. The 200 seeds are 270920260–270920459; matching seeds pair configurations within each opponent group, not their subsequent trajectories.

We report episode-level means and 95% bootstrap intervals using 10,000 resamples, and paired differences by resampling complete seed-matched episodes. Survival and self-destruction proportions also receive Wilson intervals; fractional score-leader share uses the bootstrap. Twenty episodes give particularly coarse estimates: two self-destructions correspond to 10%, but the 95% Wilson interval is approximately 2.8–30.1%. The intervals are pointwise, without correction across the exploratory component contrasts. Evaluation uncertainty is a separate issue from

variability across independently trained networks [1, 2]. With only one frozen trained artifact in this experiment, conclusions concern that checkpoint, not a reliable ranking of learning algorithms.

The mask audit checks candidate actions independently of weights; the frozen experiment measures selected actions and competitive outcomes. Neither estimates sample efficiency under a different training mask.

5 Training

Main authors: Mark Salzmann, Jesper Eggers

5.1 Exploration with an epsilon-greedy policy

Main author: Mark Salzmann

Both agents implement masked ϵ -greedy selection. Let $\mathcal{A}_m(s)$ be the actions retained by its mask. During training, the action is sampled as

$$a_t = \begin{cases} \text{a uniform random action from } \mathcal{A}_m(s_t), & \text{with probability } \epsilon_t, \\ \arg \max_{a \in \mathcal{A}_m(s_t)} Q_\theta(s_t, a), & \text{with probability } 1 - \epsilon_t. \end{cases} \quad (11)$$

Exploration supplies experience with actions that the current network may undervalue, whereas exploitation uses what has already been learned. Both branches respect the mask. The random branch can also select the greedy action, so ϵ_t is the probability of random selection, not necessarily the probability of taking a non-greedy action.

In order to provide sufficient exploration early in training, but gradually reduce it as the agent learns, we use an exponentially decaying epsilon

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\text{start}} - \epsilon_{\min}) \cdot \exp\left(-\frac{t}{T_{\text{decay}}}\right). \quad (12)$$

It is beneficial to choose $T_{\text{decay}} = \frac{T_{\text{Half-life}}}{\ln 2}$.

During the whole training process, the agent is faced with different training epochs. This introduces varying scenarios, obstacles, and opponents. The RUEHL curriculum reheats exploration at stage boundaries, as discussed in Section 5.2.1. The final agent instead derives exploration from its stored step counter, as described below.

5.2 RUEHL curriculum and optimization

Main author: Mark Salzmann

5.2.1 Training curriculum

Main author: Mark Salzmann

The training progression considered in this project starts with navigation, then introduces bomb use, and finally adds opponents. This curriculum is motivated by the difficulty of discovering all three behaviors at once: an inexperienced agent can otherwise die before collecting useful examples of delayed rewards. Table 10 links the progression to the available scenarios.

Scenario	Crate density	Coins	Role in the curriculum
<code>coin-heaven</code>	0	50	Learn movement and collection; the initial stage is intended to run without opponents.
<code>loot-crate</code>	0.75	50	Learn to open routes, reveal coins, and escape after placing bombs.
<code>classic</code>	0.75	9	Combine collection and bomb use with competition against opponents.

Table 10: Configured scenarios and their intended training roles. Crate density is the placement probability before walls and starting areas are cleared. Opponents are configured separately from the scenario.

Stage 1: coin collection. In the `coin-heaven` scenario, visible coins give a relatively direct objective: move through the maze and collect them efficiently. The agent can learn spatial relationships between its position, walls, and rewards. For this initial stage of training, we started the epsilon greedy strategy with $\epsilon = 1$, ensuring maximum exploration. Via reward shaping, the agent receives immediate feedback for collecting coins, and negative feedback for undesirable actions, guiding it towards efficient coin collection.

Stage 2: crate destruction. The `loot-crate` scenario adds destructible obstacles and places coins preferentially underneath them. Successful collection can now require a sequence of actions: approach a crate, place a bomb, reach safety, and return to collect a revealed coin. This stage introduces delayed credit assignment and the conflict between making progress and avoiding self-destruction. Initializing from the pretrained Network on `coin-heaven`, we continued training with substantial epsilon reheating $\epsilon = 0.85$ to encourage exploration in the presence of destructible crates. For this stage, a different reward profile is used, emphasizing successful crate destruction and safe bomb placement, while still rewarding coin collection and very importantly, penalises suicidal actions.

Stage 3: opponents and competitive play. The main goal for this stage is to adapt to less placed coins, while dealing with dynamic opponents. A progression from simpler opponents to stronger ones would let the agent adapt these skills gradually. For this purpose, we iteratively improved the model, by transferring the pretrained network from `loot-crate` to `classic` with slight epsilon reheating $\epsilon = 0.20$, then gradually increased the difficulty of opponents.

5.2.2 Optimization and training state

Main author: Mark Salzmann

The training loop stores transitions in a replay buffer of fixed size, and samples minibatches uniformly without replacement, starting after the warm-up period. It implements the Double DQN target from Equation (5): the masked online network selects the next action, and the target network evaluates it. For optimization, the `ruehl_based_agent` uses AdamW with a smooth L1 loss, and clips the gradient norm to 5.0. The target network receives a hard copy of the online weights every `TARGET_UPDATE` gradient updates.

The replay capacity, batch size, discount factor, learning rate, warm-up, update intervals, and evaluation budget are read from `Hyperparams.prm`.

5.2.3 Evaluation and model selection

Main author: Mark Salzmann

After `EVAL EVERY TRAINING ROUNDS` training rounds, the agent plays `EVAL ROUNDS` greedy evaluation rounds with $\epsilon = 0$. During these rounds, no transitions are added to the replay buffer and no optimization or exploration decay steps occur. Periodic evaluation ensures that model selection is based on up-to-date performance rather than outdated checkpoints, especially for DQN agents [4]. For coin collection and crate exploration, the selection is based on the actual game score rather than accumulated shaping rewards. In competitive play, the selection score is extended by penalising self-destruction and favoring eliminating opponents.

5.3 Final-agent optimization and checkpointing

Main author: Jesper Eggers

The current final-agent implementation uses uniform replay with capacity 100,000, batch size 64, and a 4,096-transition warm-up. AdamW minimizes Smooth L1 loss at learning rate 10^{-4} and weight decay 10^{-5} , with discount 0.98 and gradient norm clipped to five. Four optimization passes occur every four regular environment steps; ten further passes occur at round end. The target is copied every 2,048 regular training-transition steps. Terminal transitions suppress bootstrapping. The network is saved after each round, so the default artifact is a continuation checkpoint rather than a best-of-validation selection. Its metadata records 1,243 logged rounds. Exploration is $\epsilon_t = 0.05 + 0.95 \exp(-t/569840)$, with the step counter restored on continuation; evaluation sets exploration to zero.

6 Experiments and Results

Main authors: Mark Salzmann, Eric Flämig, Jesper Eggers

6.1 Historical RUEHL selection summaries

Main author: Mark Salzmann

Table 11 reports the historical best-model summaries for each curriculum stage. The configured checkpoint-selection batch contains 20 greedy evaluation rounds. These selected validation summaries are distinct from an independent test of the frozen checkpoint. The first row gives declared round budgets; command-line budgets in the RUEHL schedule include evaluation rounds.

Metric	Coin collection	Loot crate	Peaceful killer	Classic killer	Classic self-play	Classic hard opponents
Declared round budget	30,000	60,000	120,000	30,000	30,000	18,500
Mean game score	50.00	44.90	6.45	7.95	7.10	3.95
Mean coins collected	50.00	44.90	3.20	4.20	2.60	1.95
Mean opponents killed	0.00	0.00	0.65	0.75	0.90	0.40
Completion rate (%)	100	20	0	0	0	0
Suicide rate (%)	0	5	20	5	10	10
Mean crates destroyed	0.00	43.40	44.05	44.60	36.35	29.10

Table 11: Reported historical best-model summaries across curriculum stages. Completion requires collecting all configured coins with no opponents remaining, and does not measure survival.

Figure 1 compares selected-checkpoint summaries for the peaceful-killer and classic-self-play stages. Official game score differs from the killer-profile selection objective $J = \bar{S} + 5\bar{K} - 5p_{\text{self}}$, where $\bar{S}$ is mean official score, $\bar{K}$ is mean credited kills, and p_{self} is the self-destruction fraction. The selection objective therefore gives an additional weight to kills already included in official score. The distinct stages and training histories prevent a controlled comparison of these values.

Selected endpoints describe achieved validation scores rather than a learning trajectory. Stage-specific scenarios, training histories, and selection objectives differ, so these summaries cannot identify convergence rates or a causal effect of the curriculum alone.

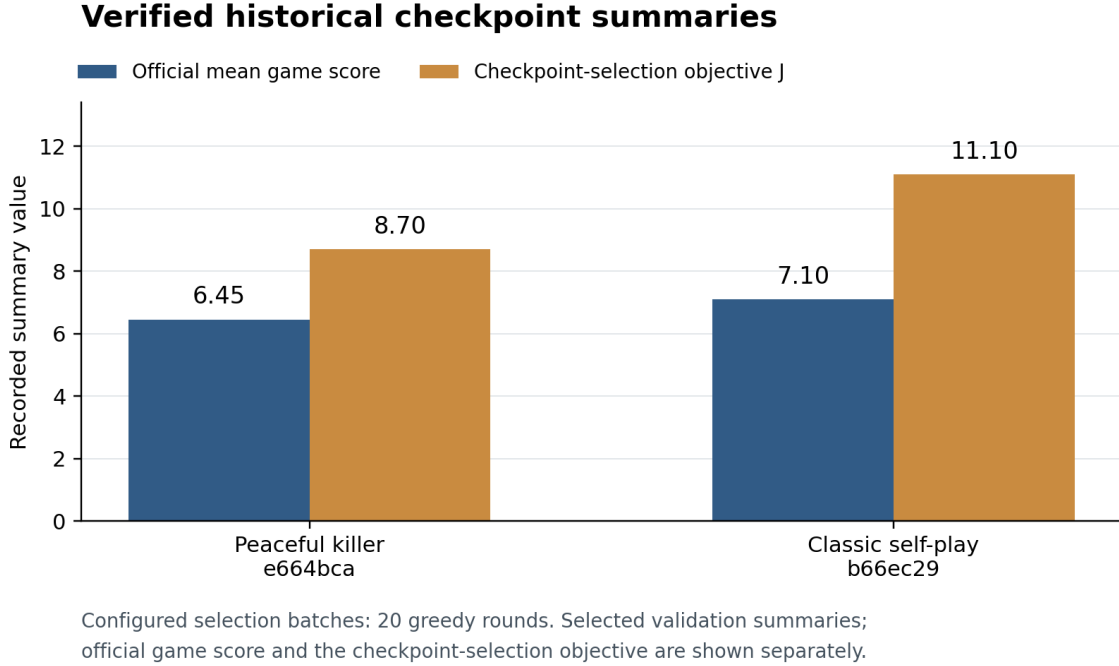


Figure 1: Historical selected-checkpoint summaries from Git revisions `e664bca` and `b66ec29`. Each snapshot configures 20 greedy selection rounds. Official mean score and the checkpoint-selection objective J are shown separately. These are validation summaries for two curriculum stages, rather than independent test estimates.

6.2 Illustrative final-agent training run

Main author: Eric Flämig

The final agent was not the submitted agent for the competition, but was trained in parallel to the Ruehl based agent. The following shorter run illustrates its training behavior; it does not establish comparative playing strength.

The following two plots show the performance of the final agent as it was trained on 300 games of loot crate with 50 coins without an opponent, followed by around 600 games of classic with nine coins and with three rule based agents as opponents. In order to determine whether or not a completely new network would be able to improve, the rolling average of the coins collected, the opponents killed, the crates destroyed, the score, the suicide ratio and the steps until the game ends were calculated over the last 10 games. Figures 2 and 3 show these plots.

As can be seen, during the training in loot crate, the network made constant improvements over the duration of the 300 rounds. The plots showing the classic games, however show that the network had significant trouble adapting to a much more complex training environment. Despite double the training and the previous knowledge, from the loot crate training, the score only shows a very slight upwards trend and no improvement in the kill-ability.

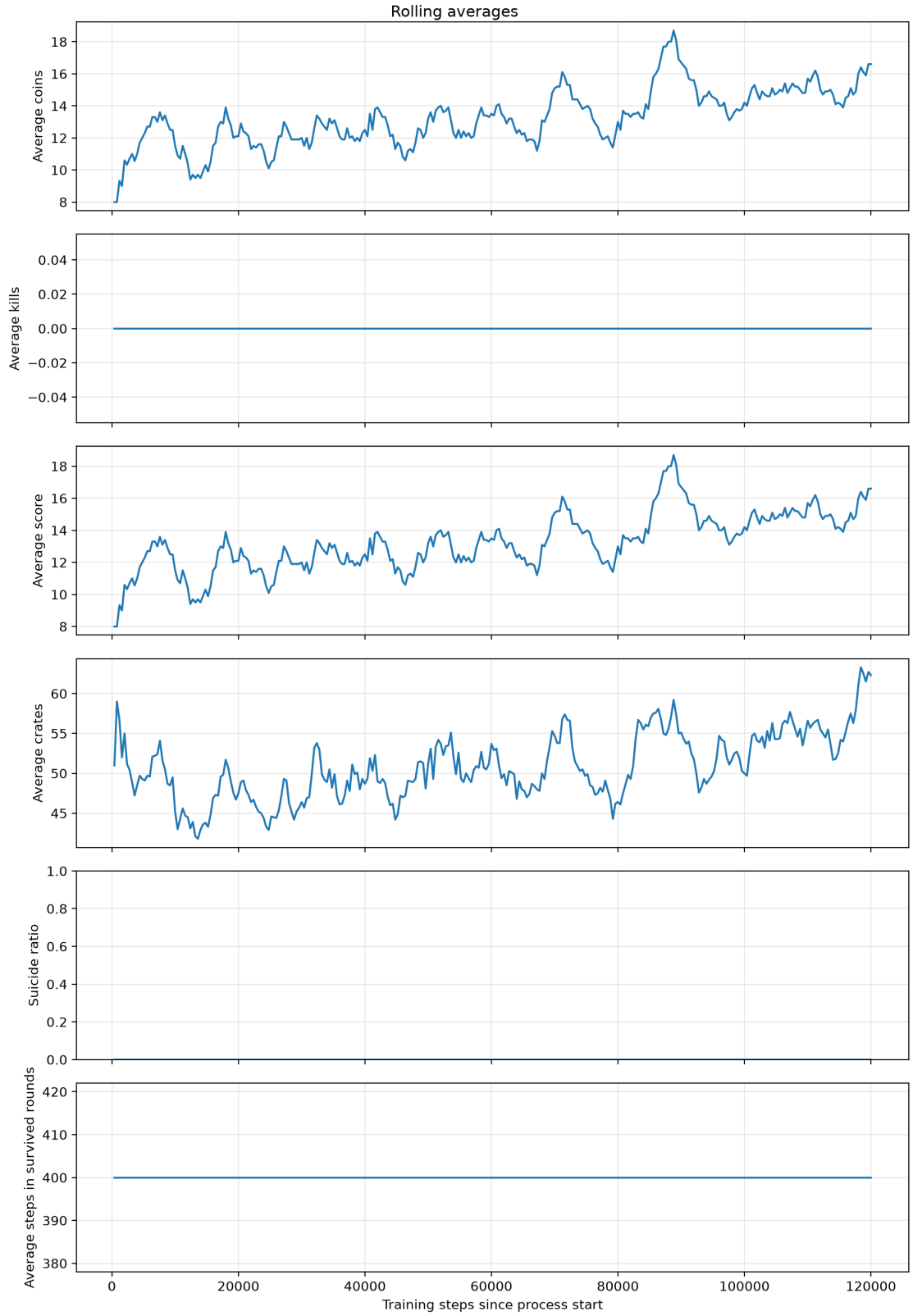


Figure 2: Archived final-agent training statistics in loot-crate without opponents. Curves are ten-round rolling averages during learning, rather than independent frozen-policy evaluations.

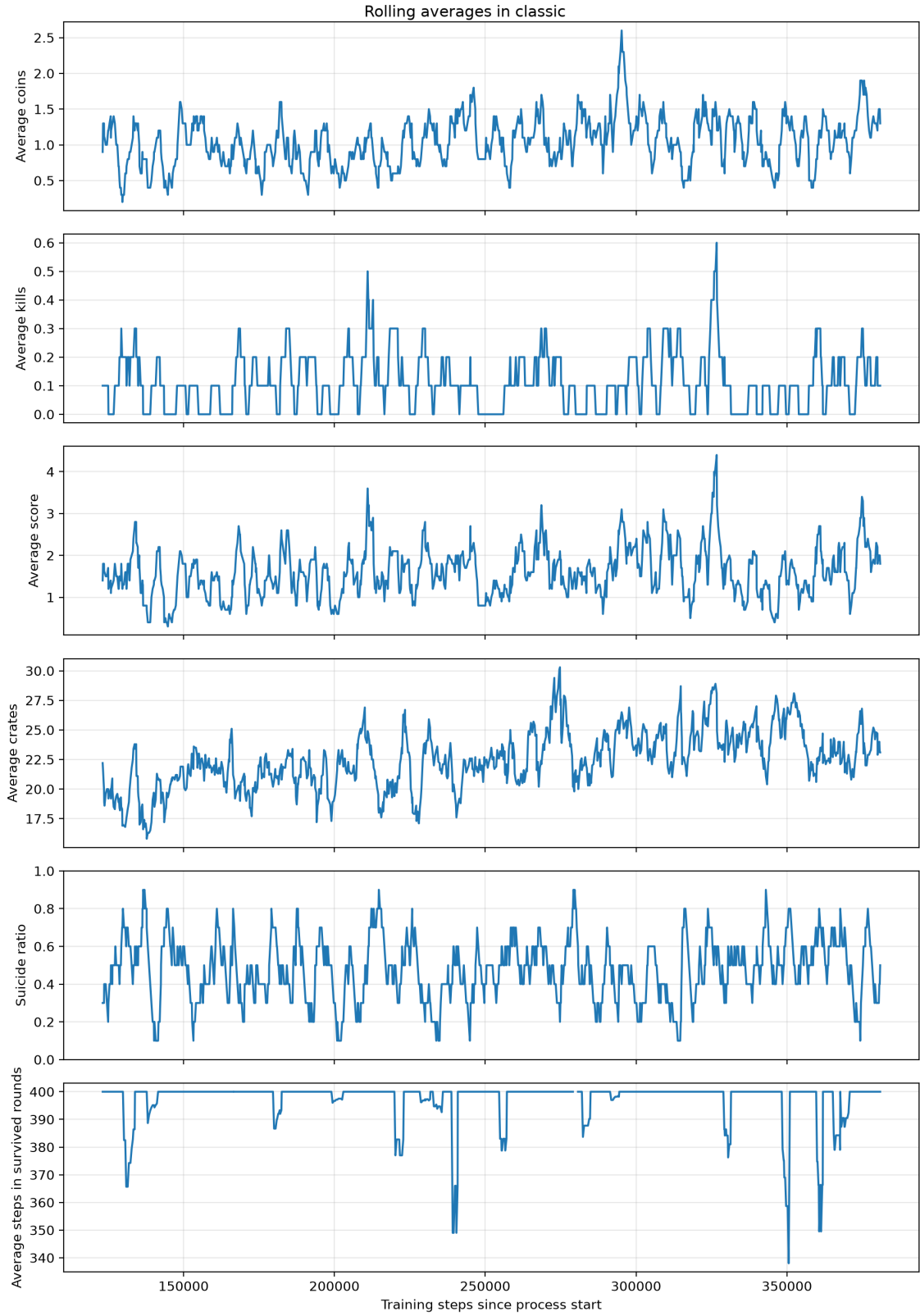


Figure 3: Archived final-agent training statistics in classic with three rule-based opponents. The final panel averages steps only over survived rounds; it is not average episode duration.

Figure 3 contains rolling averages of behavior during training, whereas Figure 1 contains selected greedy-evaluation summaries. These are different estimands. The agents also use different representations, masks, rewards, and training budgets. The early loot-crate improvement is an observation of this final-agent run, and does not isolate the effect of network size or establish faster convergence than RUEHL. During development, final-agent rounds were reported to sometimes exceed 60 seconds, with approximately 30 seconds often observed. These development observations concern complete training rounds, which include simulation and optimization as well as action selection. They do not isolate the costs of the network or action mask. Complete decision latency is a separate deployment quantity from elapsed training time per round.

The short classic trace does not determine whether adaptation is limited by the representation, the action mask, reward shaping, or training exposure. A slight upward trend cannot predict that equal training would match or exceed RUEHL’s performance. Frozen-policy comparisons measure competitive outcomes; matched training experiments would be needed to attribute differences to individual design choices.

6.3 Engine-based diagnosis of action masking

Main author: Jesper Eggers

6.3.1 Question and experimental construction

Main author: Jesper Eggers

Both agents restrict the actions available to their Q-networks. This restriction also enters the Double DQN bootstrap target, so the mask affects exploration, evaluation, and the value function being learned. A useful mask must balance two errors: admitting actions that make survival impossible and excluding actions that permit escape. A restrictive mask need not achieve the latter balance better. We tested whether the final agent’s temporal search improves this balance relative to RUEHL’s geometric and immediate-danger checks.

We generated 3,000 states, equally divided among crate-placement probabilities 0, 0.30, and 0.75 on the configured 17×17 board. The focal agent occupies a randomly selected free tile; one to three other bombs have timers sampled from zero through three. Approximately one quarter of states also contain a bomb beneath the focal agent. Nearby bomb placements are oversampled to expose local hazards. All generation parameters and the fixed seed are retained in the supplementary scripts.

These are stress-test states, not observations sampled from either policy. They contain no opponents, visible coins, or initially active explosions, and no new bombs appear after the candidate first action. The test measures conditional escape feasibility, including a hypothetical own bomb for BOMB; it does not measure attack quality or how frequently these hazards arise during competitive play.

6.3.2 Reference calculation and metrics

Main author: Jesper Eggers

For each state and first action, the reference asks whether any legal continuation survives six environment steps. A reachability search propagates possible positions through terrain, bomb occupancy, and lethal tiles computed with the engine’s actual `update_explosions` and `update_bombs` methods. In this engine, blasts pass through destructible crates and stop at stone

walls. Destroyed crates become traversable on the following action step. Waiting on a bomb beneath the agent remains legal; moving back onto an occupied bomb tile does not.

A timer-zero bomb explodes after the next action; timer one explodes after the second action. The blast is lethal on its creation step and the following action step. A newly placed bomb also decrements during placement; six steps cover its complete lethal period without further bombs. Assertions verify timing, terrain, and the distinction between lethal explosions and harmless smoke. All 265 surviving witness paths replayed through the engine executed legally and avoided lethal explosions.

Of the generated states, 2,453 admit at least one surviving movement or waiting continuation. We compute the principal diagnostics on these recoverable states, so a mask is not penalized where every choice is already doomed. An admitted action is unsafe when the reference finds no surviving continuation. The unsafe-admission fraction divides these errors by all admitted actions in the relevant category. The exclusion fraction divides viable movement and waiting actions rejected by the mask by all viable movement and waiting actions. Bombs are assessed separately: usefulness constraints can correctly exclude a survivable but strategically pointless placement. Actions from the same state are correlated, so these fractions describe this benchmark rather than independent Bernoulli trials.

Diagnostic	RUEHL mask	Final-agent mask
Unsafe admitted movement/wait	84/6103 (1.38%)	1/6367 (0.02%)
Unsafe admitted bombs	53/712 (7.44%)	0/669 (0.00%)
Viable movement/wait excluded	350/6369 (5.50%)	3/6369 (0.05%)

Table 12: Measured diagnostics on 2,453 recoverable states from the 3,000-state suite. Each fraction uses its own candidate-action denominator.

6.3.3 Measured results and failure mechanisms

Main author: Jesper Eggers

RUEHL admits 84 unsafe movement or waiting actions and 53 unsafe bomb placements; the final mask admits one and zero, respectively (Table 12). RUEHL excludes 350 viable movement or waiting actions, compared with three for the final mask. The final mask improves both diagnostics here, showing that its longer-horizon constraints need not retain fewer viable actions.

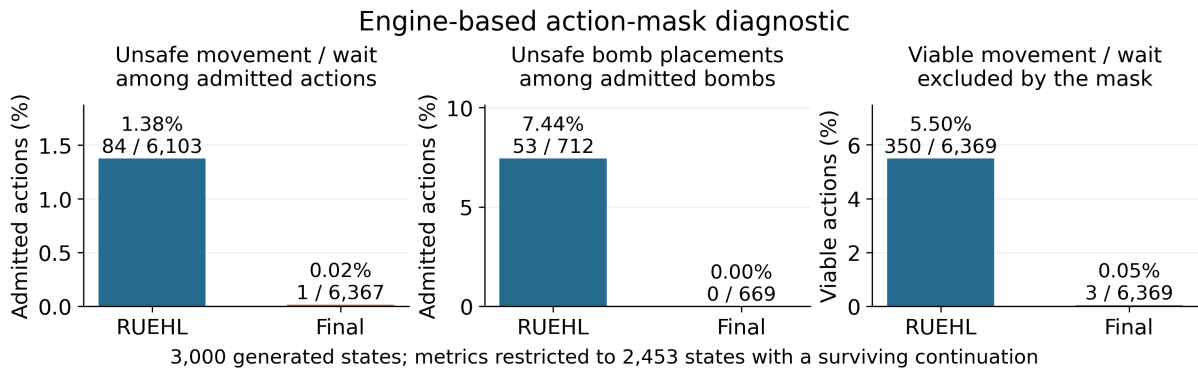


Figure 4: Observed action-level diagnostics from the generated suite. Counts and denominators are shown explicitly; these are not measured gameplay death rates.

The counterexample in Figure 5 explains how these errors can interact. The agent stands at (9, 8), while a bomb at (9, 6) has timer one. Moving down to (9, 9) and then right to (10, 9) escapes

before the existing bomb explodes. RUEHL excludes the initial downward move because its immediate-danger set already includes the blast of timer-one bombs. Nevertheless, its separate bomb-placement test finds a geometric route outside the proposed own blast without jointly checking the existing bomb’s future timing. The resulting mask admits only BOMB. The reference finds no surviving continuation for that action, and the final-agent mask instead admits DOWN. Here, the network cannot correct the mistake: the mask removes the viable choice before Q-values are compared.

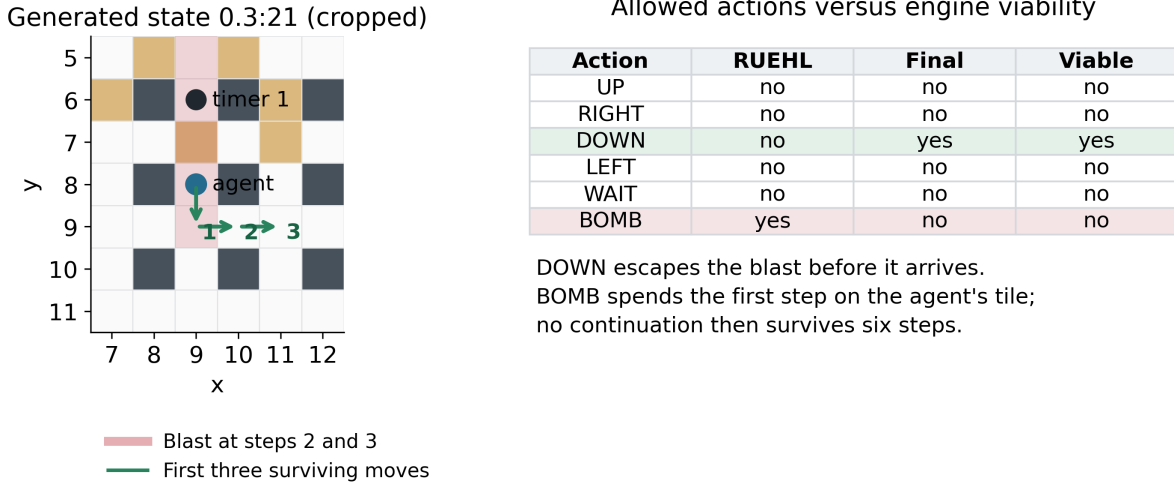


Figure 5: A recorded generated counterexample, not a tournament replay. The board is cropped; all bombs on the full board enter the reference calculation. Green arrows show the first three moves of a surviving continuation beginning with DOWN.

The final mask’s residual unsafe admission occurs in state 0.75:428. Its search keeps crates permanently blocking, missing a route opened by an existing bomb. All movement and waiting searches fail, so the fallback restores physically legal movement, including an unsafe upward move and viable leftward move. Predicting terrain updates would address this implementation defect.

6.3.4 Implications and limits

Main author: Jesper Eggers

The diagnostic establishes that the temporal mask better preserves six-step escape possibilities under the specified hazard evolution. It also exposes a repairable implementation defect: static terrain can trigger a permissive fallback despite a dynamically available escape.

A viable continuation is an existential result. The network may select different later actions, and opponent movement or new bombs may invalidate the path. Consequently, generated-state accuracy does not directly predict competitive survival. The following experiment measures deployment consequences by holding final-network weights fixed and crossing learned ranking with the two masks. Claims about learning would additionally require consistent next-state masking in the Double DQN target and matched retraining. These diagnostic results establish constraint quality under the reference dynamics, rather than a ranking of complete trained agents.

6.4 Frozen-policy performance and opponent dependence

Main author: Jesper Eggers

Table 13 reports all eight evaluated configuration–opponent combinations, with 200 games each. The native final policy scores 1.945 points per game against three rule-based opponents, with a 95% interval of [1.655, 2.250]. Its mean comprises 1.295 coins and 0.130 credited kills; multiplying the latter by five accounts for the remaining 0.650 points. Survival is 45.0%, while self-destruction occurs in 54.0% of games. Of 110 deaths, 108 include self-destruction credit, identifying frequent exposure to the agent’s own blasts. Repeated bombing and a strong safety diagnostic do not by themselves establish a strong competitive player.

Against the mixed population, native mean score rises to 2.640 [2.285, 3.015], comprising 1.415 coins and 0.245 kills. The 0.695-point difference from the rule-based population consists of 0.120 additional coin points and 0.575 additional kill points. This decomposition identifies the source of the observed score change, without attributing it to an unmeasured learning mechanism. Survival rises to 51.5% and self-destruction falls to 44.0%. These are results for one frozen artifact facing two specified populations, rather than a universal measure of opponent difficulty.

Relative rank gives a different perspective. The native agent’s tie-aware score-leader share is 15.4% [10.8, 20.2] against three rule-based opponents and 10.2% [6.5, 14.5] against the mixed population. The higher absolute score in mixed games therefore does not translate into a higher observed share of first places. Changing opponents also changes the scores the focal agent must exceed. This rank comparison is descriptive; the paired component tests are conducted within each population. The results show why official return and relative success should both be reported.

Configuration	Games	Score [95% CI]	Coins	Kills	Self-destruction (%)	Survived (%)	Leader share (%)
<i>Three rule-based opponents</i>							
Final Q + temporal mask	200	1.95 [1.66, 2.25]	1.295	0.130	54.0	45.0	15.4
Final Q + RUEHL mask	200	0.57 [0.42, 0.73]	0.520	0.010	90.5	3.0	2.4
Random + temporal mask	200	1.09 [0.85, 1.34]	0.590	0.100	43.0	43.5	5.4
Random + RUEHL mask	200	0.15 [0.10, 0.21]	0.150	0.000	97.0	0.5	0.0 [†]
<i>Mixed opponent population</i>							
Final Q + temporal mask	200	2.64 [2.29, 3.02]	1.415	0.245	44.0	51.5	10.2
Final Q + RUEHL mask	200	0.51 [0.39, 0.64]	0.480	0.005	98.0	1.0	0.5
Random + temporal mask	200	1.14 [0.91, 1.39]	0.665	0.095	31.5	63.0	2.0
Random + RUEHL mask	200	0.15 [0.10, 0.21]	0.155	0.000	98.5	0.0	0.0 [†]

Table 13: Episode means from 200 games per row; score brackets are 95% episode-bootstrap intervals. Other columns are point estimates, with intervals retained in the supplementary JSON. Score-leader share splits credit among tied highest final scores, including dead agents. Inference is conditional on one checkpoint and the stated opponents. [†]The degenerate all-zero bootstrap interval is suppressed; zero score-leading episodes among 200 games gives a Wilson upper bound of 1.9% on leader presence, which also bounds expected fractional share.

With the same temporal mask, random selection scores 1.090 against rule-based opponents and 1.140 against the mixed population. Learned ranking adds 0.855 points [0.490, 1.220] and 1.500 points [1.080, 1.920], respectively, in paired comparisons. For rule-based opponents, 0.705 of the advantage comes from coins and 0.150 from kills. In mixed games, coins and kills contribute 0.750 points each. The improvement is therefore measurable beyond the hand-designed constraints, with its composition depending on the opponents.

Higher score does not imply lower risk. Learned temporal selection increases self-destruction over its random control by 11.0 percentage points [2.0, 20.5] against rule-based opponents and

12.5 percentage points [3.5, 21.5] in mixed games. Its mixed-game survival decreases by 11.5 percentage points [2.5, 20.5]. The trained ordering is more productive under the official score while retaining a substantial safety weakness. Choosing a policy solely from survival would miss its scoring benefit; choosing solely from score would conceal the remaining self-inflicted losses.

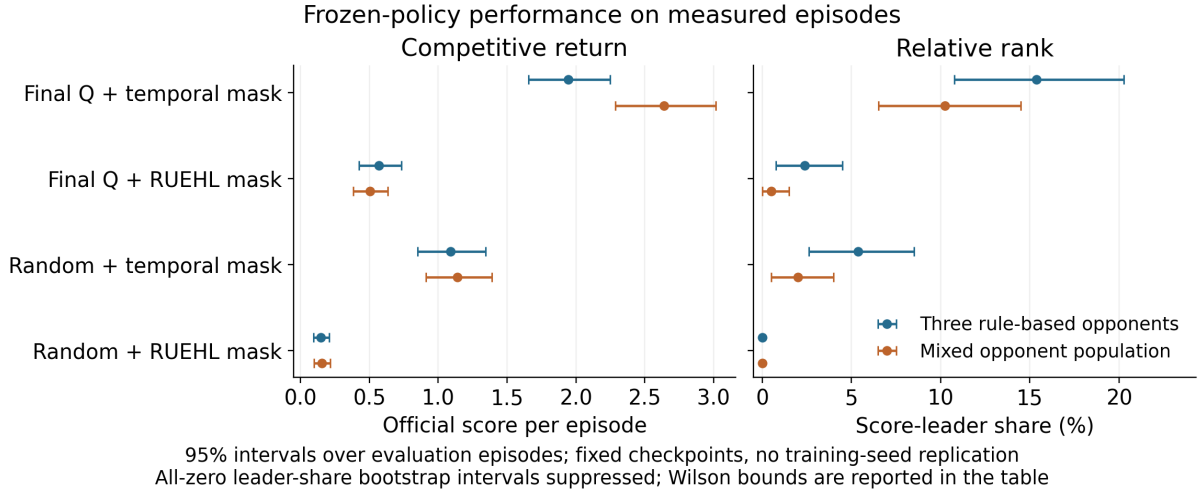


Figure 6: Official return and score-leader share for the four frozen decision configurations and two opponent populations. All learned rows use the same final-network weights.

6.5 What the learned ranking and safety mask contribute

Main author: Jesper Eggers

The aggregate outcomes suggest benefits from both constraints and learned selection. We now quantify their separate deployment effects and whether those effects interact. The four configurations form a controlled 2×2 deployment experiment. Writing $\mu_{q,m}$ for mean official score under ranking rule q and mask m , the temporal mask’s effect with the learned network is $\Delta_Q = \mu_{Q,T} - \mu_{Q,R}$. Its effect under uniform action selection is $\Delta_{\text{random}} = \mu_{\text{random},T} - \mu_{\text{random},R}$. The difference $I = \Delta_Q - \Delta_{\text{random}}$ tests whether the mask’s measured effect depends on the ranking rule. We resample the four results for each seed together to preserve the paired design. The interaction captures their joint operation: filtering actions changes the preference expressed and the subsequent trajectory.

For learned selection, replacing the RUEHL mask with the temporal implementation adds 1.375 points [1.090, 1.670] against rule-based opponents and 2.135 [1.775, 2.505] against the mixed population. Self-destruction decreases by 36.5 and 54.0 percentage points, respectively. The random controls also benefit: their score gains are 0.940 [0.715, 1.185] and 0.985 [0.760, 1.230]. This supports a deployment benefit from the temporal-mask implementation even under random ranking.

The score interaction is 0.435 [0.075, 0.805] for rule-based opponents and 1.150 [0.715, 1.600] for mixed opponents. In both populations, the observed temporal-mask benefit is larger with learned selection. The components therefore complement each other in this experiment, although the smaller interaction’s interval approaches zero and remains a pointwise, exploratory comparison.

Within a fixed mask, learned-versus-random selection measures the deployed value of trained preferences. Masked random controls retain legality, danger, and bomb-usefulness checks. An advantage over them demonstrates value beyond these constraints, without implying that the

network learned the safety rules independently. Random deployment does not recreate an otherwise identical agent trained without learning.

The frozen network was trained with the temporal mask. Swapping to RUEHL’s mask changes both its immediate choice set and the states subsequently encountered. Distribution shift is therefore part of the tested deployment intervention, rather than a nuisance removed by holding weights fixed. The paired contrast supports a causal statement about this implementation under these opponents; it cannot establish the performance of RUEHL’s own trained network or the sample efficiency of either mask during training. A stronger learning claim requires matched retraining budgets, representations, rewards, and multiple initialization seeds.

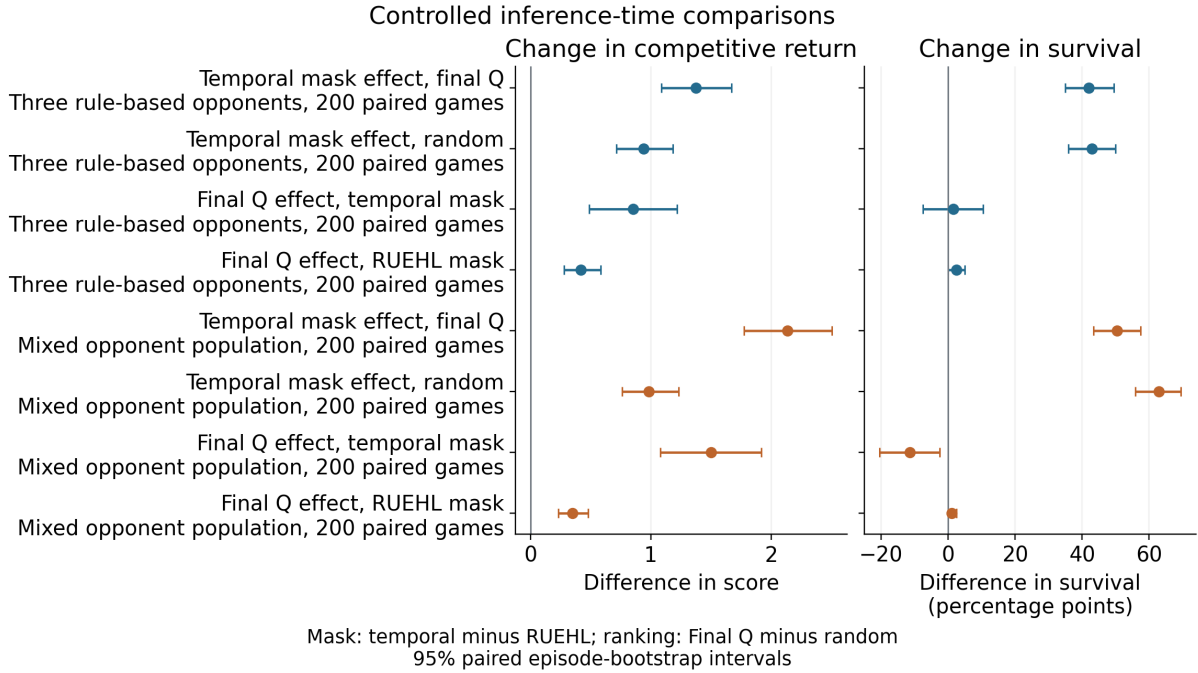


Figure 7: Paired deployment effects on score and survival. Mask effects subtract RUEHL from temporal masking; ranking effects subtract uniform random from Final Q selection within a fixed mask. Whole seed-matched episodes are resampled together.

6.6 Why predicted safety does not ensure competitive survival

Main author: Jesper Eggers

We audit selected actions at indices 1, 11, 21, and so forth using the same engine reference, augmented with observed active explosions and opponents blocking the first move. Future opponent actions and new bombs remain unspecified. Against rule-based opponents, native selection has zero unsafe choices among 5,401 recoverable observations, whereas the learned RUEHL-mask variant has eleven among 978. All eleven precede observed death within six steps. This is a descriptive association; the sampling scheme and omitted opponent dynamics prevent interpreting the reference as a complete attribution of deaths.

To investigate the native policy’s losses, we replay its first episode, seed 270920260, and audit every action. All 216 requested actions and official outcomes match the primary run. Of the 216 observed states, 215 are recoverable; the reference finds a surviving continuation after each selected first action in those states. The final state is already unrecoverable. Figure 8 shows why. At step 212, the agent places a bomb at (8, 15). At step 215, it occupies (10, 15) and requests

RIGHT, with a conditional escape through (11, 15). An opponent at (11, 14) moves down into that destination before the focal action executes. The movement becomes invalid. At step 216, the focal agent remains at (10, 15), its bomb has timer zero, and the opponent occupies the intended escape tile. The fallback selects LEFT inside the imminent own-bomb blast, followed by self-destruction. At step 215, LEFT also had a conditional escape. Binary viability misses the routes' different exposure to interference; neither route establishes a counterfactual winning policy.

Attack productivity is also limited. Native rule-based games contain 3,706 dropped bombs and 26 credited kills. Of 1,314 placements threatening an opponent's current tile, only twelve satisfy the implementation's predicted-trap test. Threatening a location rarely establishes a predicted trap here. These are placement diagnostics, not verified trap outcomes, and kills per bomb is not a success probability. Improving opponent-aware escape and attack reasoning is therefore a more specific hypothesis than simply increasing network size.

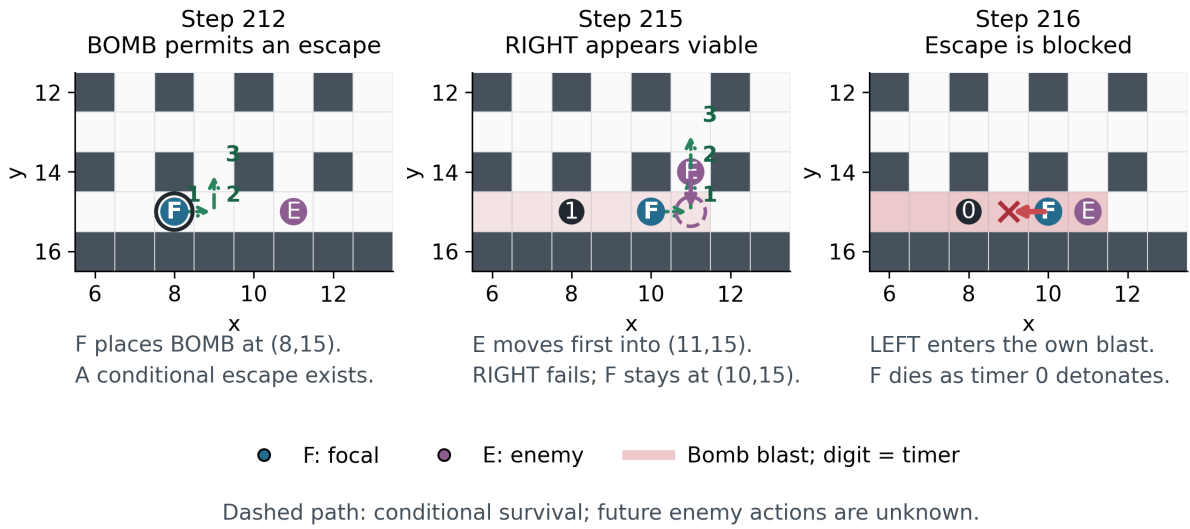


Figure 8: Recorded states from the first native-policy episode. Opponent movement blocks a requested escape before it executes. Conditional reachability and physical execution consequently diverge.

6.7 Computational constraints and reproducibility

Main author: Jesper Eggers

The instantiated RUEHL and final networks contain 383,559 and 1,804,487 parameters, respectively. Parameter count alone does not determine deployment cost because feature construction and survival searches also execute for every action. We therefore time the complete callback on an Apple M4, using CPU inference with one PyTorch thread per evaluator process. Several evaluations run concurrently; these are local measurements without exclusive processor access. Observer diagnostics execute outside the timed callback, while the engine retains its actual timeout enforcement.

Across 266,007 callbacks, no timeout or timeout skip occurs. Against rule-based opponents, the native policy's median, 95th-percentile, and maximum observed latencies are 4.08, 7.61, and 101.07 ms, respectively, below the configured 500 ms allowance on this machine. Random callbacks omit feature construction and neural inference, so their latency compares complete deployment costs rather than the isolated cost of action ranking.

Figure 9 reports pooled action quantiles, rather than averages of per-episode percentiles. Its intervals resample whole episodes. The evaluated default artifact, `agent_code/final_agent/Ultra_Network_agent_saved_model_v2.pt`, has SHA-256 prefix `c268c3fcb5df3491`; the complete hash, source hashes, checkpoint metadata, opponent definitions, seeds, dependencies, and action records are retained in the experiment manifests. Reproduction uses the project’s source repository, https://github.com/ericgoldclub/bomberman_rl, and the supplementary evaluation scripts. Inference latency and elapsed training time quantify different workloads; the local measurements also do not establish compliance on the tournament’s AMD processor.

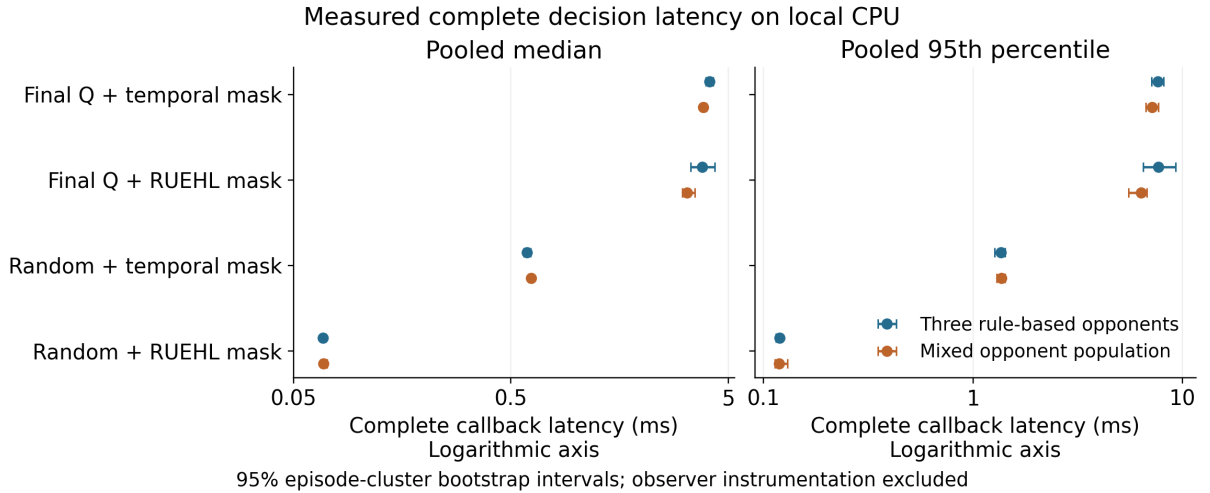


Figure 9: Complete callback latency. Learned rows include features, masking, and network inference; random rows include masking and selection. Intervals use 2,000 episode-cluster bootstrap resamples. Concurrent local CPU evaluation does not reproduce the tournament machine.

7 Conclusion

Main authors: Mark Salzmann, Eric Flämig, Jesper Eggers

7.1 Development conclusions

Main author: Mark Salzmann; Final-agent discussion: Eric Flämig

We developed and evaluated a different agents based on Double DQN, a dueling network architecture, engineered spatial and scalar features, action masking, and a staged training curriculum. The curriculum decomposed the task into progressively harder objectives. The reported selected `coin-heaven` summary collected all 50 coins, and the `loot-crate` summary recorded substantial crate destruction; these results describe the corresponding selection batches rather than guaranteeing perfect generalization. The historical RUEHL selection summaries report crate destruction, coin collection, credited eliminations, and self-destruction fractions of 5–20% in the competitive stages. These selected batches are distinct from the fresh frozen-policy experiment.

The reported selected hard-opponent RUEHL summary gives mean score 3.95, 0.40 credited eliminations, and zero completion over a configured twenty-game selection batch. These checkpoint-selection statistics are distinct from the independent component evaluation. These summaries show credited eliminations, but do not establish a competitive win rate or explain particular

opponent counter-strategies. Zero completion under the strict all-coins criterion must not be interpreted as zero survival.

Possible next steps include evaluation over longer training periods, more systematic hyperparameter tuning, and training against a diverse population of opponents. These extensions would clarify which design choices contribute most strongly to performance and may improve robustness in the full competitive setting.

The final-agent run shows improvement in the simpler loot-crate stage and limited attack success in the short classic stage. Because training histories and measurement protocols differ, this does not establish faster convergence, inferior architecture, or a common-budget advantage for either design. RUEHL was the submitted artifact, while the final agent provides a contrasting allocation of learned decisions and explicit guidance.

The controlled frozen-policy campaign measures the two masks with one verified final-agent checkpoint under common opponents and seeds. With features and weights held fixed, it identifies how the complete acting mask and learned action ranking contribute to score and survival for the final policy. Comparable retraining budgets and independent evaluation remain necessary to distinguish training exposure from changes in architecture or guidance.

7.2 Synthesis and focused next experiment

Main author: Jesper Eggers

The completed experiments support three distinct conclusions. Temporal masking preserves more viable escapes and admits fewer doomed actions in the generated suite. With final-network weights frozen, it improves score and reduces self-destruction under both tested opponent populations. Learned ranking adds further score beyond the temporal mask, but increases self-destruction relative to the corresponding random control. These results establish complementary contributions of masking and learned selection under controlled deployment conditions.

The competitive trace identifies a concrete remaining weakness: an escape can depend on a tile that an opponent reaches before movement executes. A focused next experiment would test immediate escape destinations for conflicts with reachable opponent occupancy, then compare the resulting constraint with the existing mask using frozen weights and paired games. Greater conservatism may prevent useful attacks, so score and survival must be assessed jointly. Retraining would follow only if this deployment test motivates it.

For future competitions, the framework should export initial-layout seeds, requested actions, invalid-action events, model hashes, and per-agent episode outcomes through one evaluation utility. Separate generators for board layouts and action order would make such comparisons reproducible by default.

References

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL <https://arxiv.org/abs/2108.13264>.
- [2] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11694>.
- [3] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287. Morgan Kaufmann, 1999. URL <https://ai.stanford.edu/~ang/papers/shaping-icml99.pdf>.
- [4] Melrose Roderick, James MacGlashan, and Stefanie Tellex. Implementing the deep Q-network. *arXiv preprint arXiv:1711.07478*, 2017. URL <https://arxiv.org/abs/1711.07478>. Version 1. Local source: 1711.07478v1.pdf.
- [5] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. *arXiv preprint arXiv:1509.06461*, 2015. URL <https://arxiv.org/abs/1509.06461>. Version 3. Local source: 1509.06461v3.pdf.
- [6] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003. PMLR, 2016. URL <https://proceedings.mlr.press/v48/wangf16.html>.